

universidade de aveiro



deti

departamento de eletrónica,
telecomunicações e informática

Complements of Machine Learning

2025/2026



Generative Models

Today's Agenda

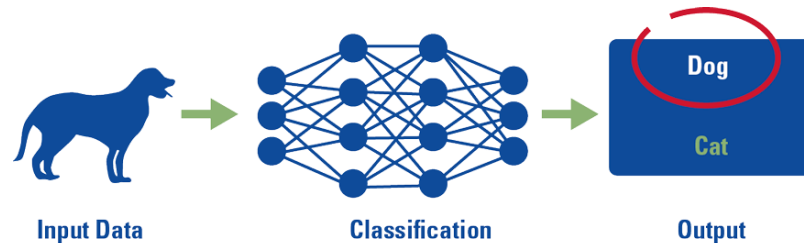
Time	Topic
13:00 - 14:15	Theory: Generative Models, VAE and GANs
14:15 - 14:30	 Break
14:30 - 16:00	Lab: DCGAN digits and Pokémon's!

1. Discriminative vs Generative Models

New class of models

What We've Done All Semester

- Until today, our deep learning journey has been focused on **prediction**.
 - We take an instance (X , like an image) and try to predict its label (Y , like "Dog").



- We are estimating **Conditional Probability: $P(Y|X)$** . (*Given this image, what is the probability it is a dog?*)
- This is a **Discriminative Model**. It uses all its computing power just to draw a decision boundary between classes.

Generative Paradigm

- What if we don't just want to classify the data? What if we want to *create* new data from scratch?

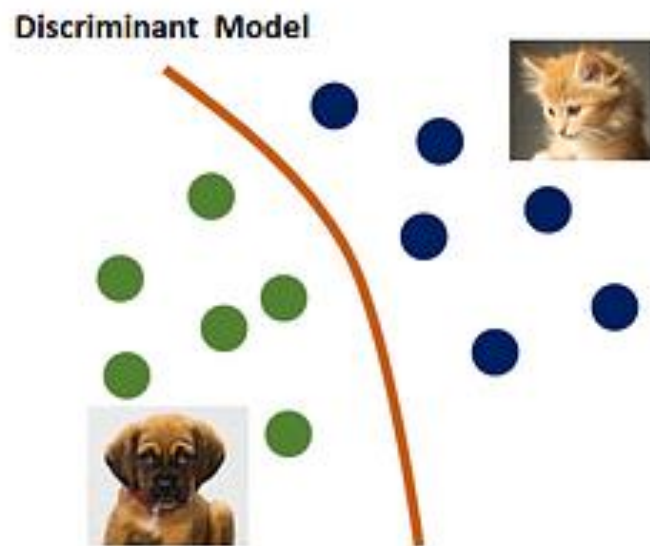
Generative Paradigm

- What if we don't just want to classify the data? What if we want to *create* new data from scratch?
- Instead of conditional probability, we model the **Joint Probability**: $P(X, Y)$ (or simply the distribution of the inputs, $P(X)$).
- This is a **Generative Model**. Instead of drawing a boundary between cats and dogs, it learns the fundamental distribution and structure of the data itself. It learns what actually makes a dog look like a dog.

Generative Paradigm

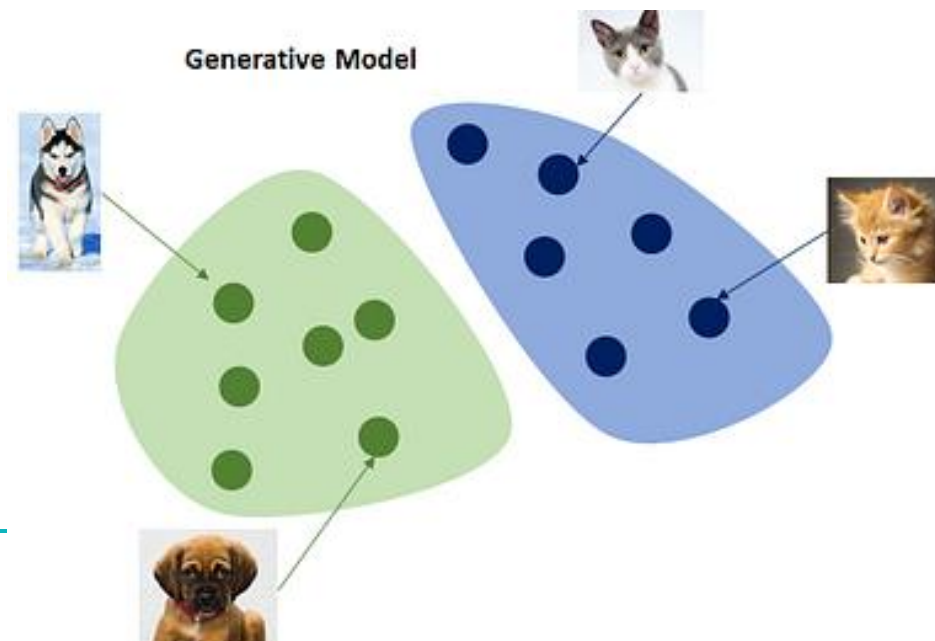
➤ Discriminative

- Draws a red line separating the green dots from the blue dots. It *does not care* how the dots got there.



➤ Generative

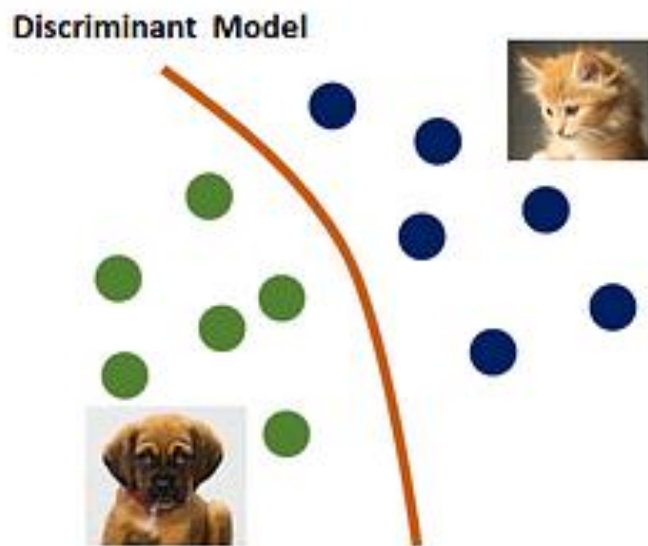
- Maps out the exact shape and location of the green cluster and the blue cluster. Because it knows the shape of the clusters, we can ask it to **generate a brand new dot** that belongs perfectly inside them.



Generative Paradigm

➤ Discriminative

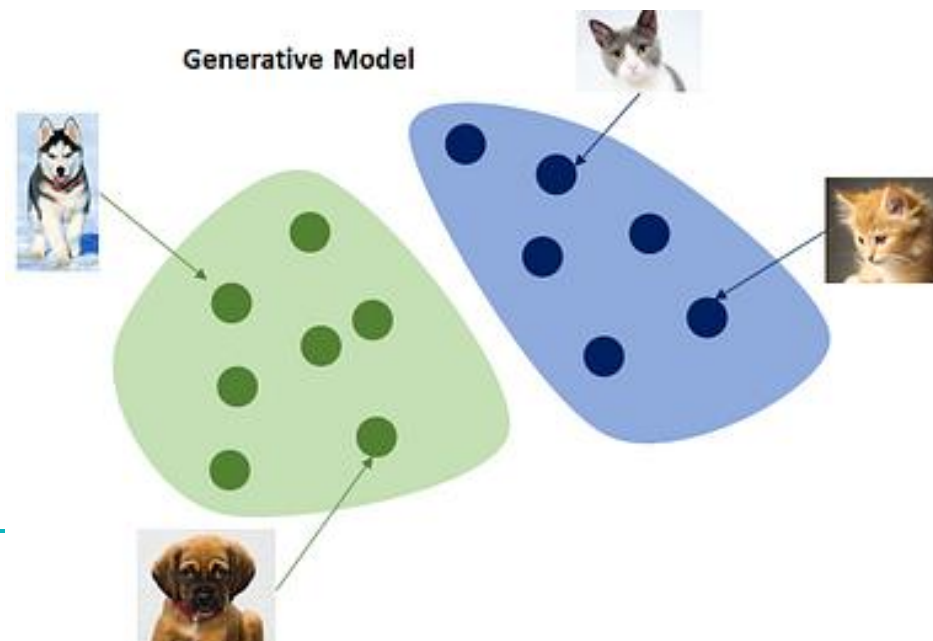
- Draws a red line separating the green dots from the blue dots. It *does not care* how the dots got there.



Which one is harder to train? Why?

➤ Generative

- Maps out the exact shape and location of the green cluster and the blue cluster. Because it knows the shape of the clusters, we can ask it to **generate a brand new dot** that belongs perfectly inside them.



Generative model as a discriminator

- Nice property of a generative model is that it can also act as a discriminator since it learns everything about the data distribution.



Generative model as a discriminator

➤ Any discriminator predicts $\arg \max_y P(Y|X)$

➤ Enter the Bayes theorem

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$$

➤ But, $P(X, Y) = P(X|Y)P(Y)$

➤ Then,

$$P(Y|X) = \frac{P(X, Y)}{P(X)}$$

$$\begin{aligned} \arg \max_y P(Y|X) &= \arg \max_y \frac{P(X, Y)}{P(X)} \\ \arg \max_y P(Y|X) &= \arg \max_y P(X, Y) \end{aligned}$$

I.e., compute all $P(X, Y)$ for all given Y , and pick the largest 😊

So Are Generative Models Just Better?

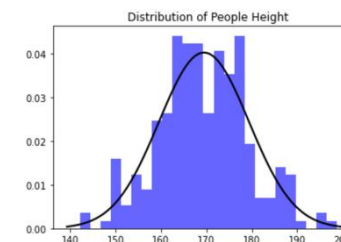
➤ No...

So Are Generative Models Just Better?

- **It's Too Hard (Computationally Expensive):** Modeling the entire universe of pixels $P(X)$ is incredibly difficult. It requires massive amounts of data and computing power.
- **Wasted Effort:** *“One should solve the problem directly and never solve a harder problem as an intermediate step.”*
 - If you just want to know if an image is a dog, you don't need a model that knows how to paint the dog's individual hairs and the grass behind it.
- **Lower Classification Accuracy:** Discriminators dedicate 100% of their "brainpower" to the boundary. Generative models spread their brainpower across the whole image. For pure classification tasks, Discriminators almost always win.

How to estimate $P(X, Y)$

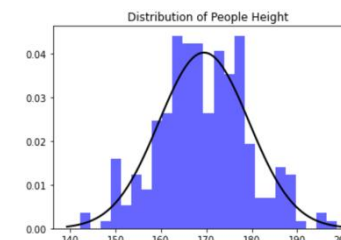
- The most traditional approach to finding a distribution is simply to count. If you want to know the distribution of human heights, you measure 1,000 people and plot a histogram.



- Why this does not work for most of the problems, like image or text?

How to estimate $P(X, Y)$

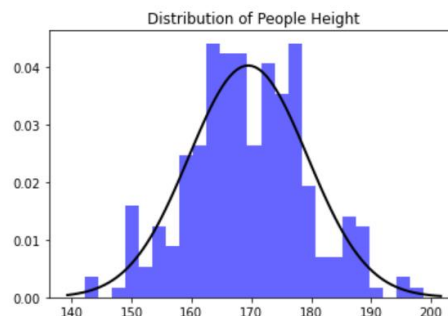
- The most traditional approach to finding a distribution is simply to count. If you want to know the distribution of human heights, you measure 1,000 people and plot a histogram.



- Why this does not work for most of the problems, like image or text?
 - Counting requires filling up bins to understand the shape of the data. High-dimensional data, like an RGB image, has hundreds of thousands of dimensions (pixels).
- **The Curse of Dimensionality**
 - The number of possible pixel combinations is practically infinite. Even with massive datasets, 99.99% of your histogram "bins" will be completely empty.

How to estimate $P(X, Y)$ (an alternative)

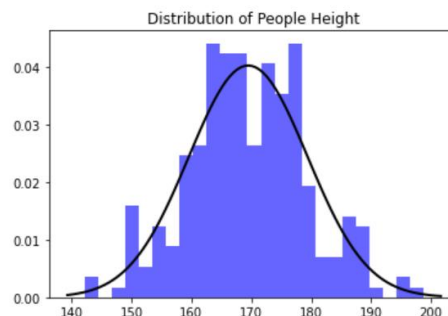
- Other approach is to just assume your data follows a known mathematical shape—like a standard Bell Curve (Gaussian distribution).
 - If you assume it's a Bell Curve, all you have to do is calculate the mean and variance.



- Why this also fails?

How to estimate $P(X, Y)$ (an alternative)

- Other approach is to just assume your data follows a known mathematical shape—like a standard Bell Curve (Gaussian distribution).
 - If you assume it's a Bell Curve, all you have to do is calculate the mean and variance.



- Why this also fails?
 - Real-world data (like the distribution of human faces) doesn't look like a neat bell curve. It has multiple peaks (multimodal), skewed tails, and bizarre, non-linear dependencies.

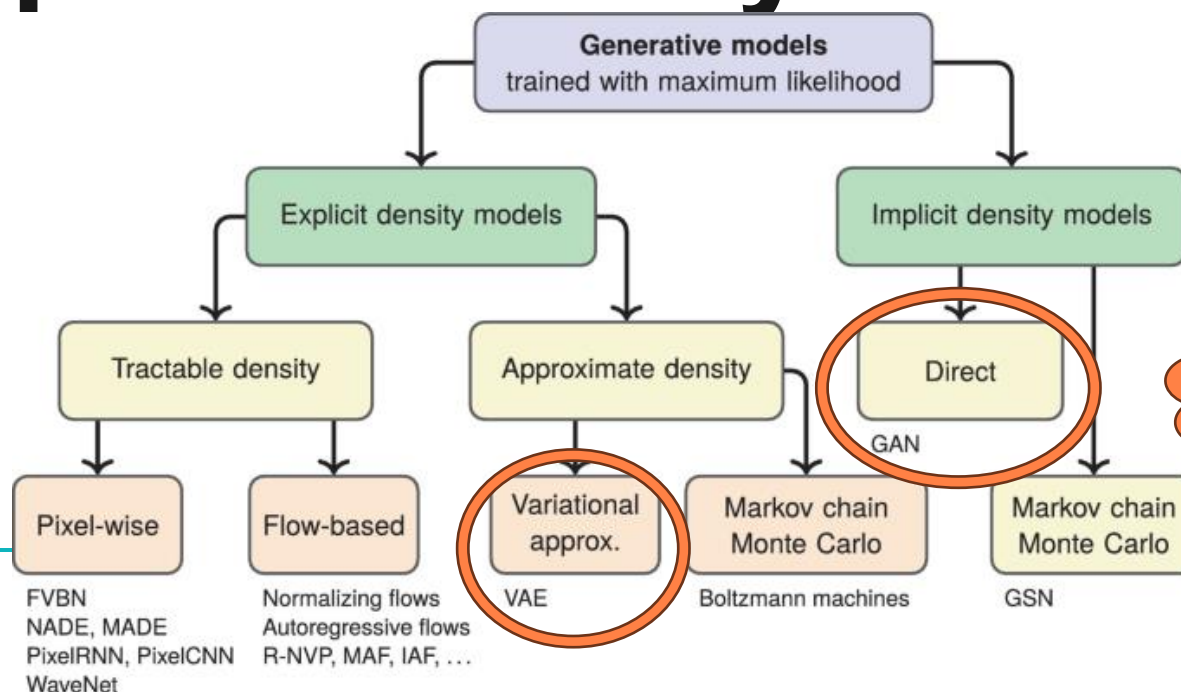
How to estimate $P(X, Y)$ – Using NN

- To map this insane complexity, we use Neural Networks to define a parameterized distribution $P_{\theta}(X)$, that maps our data into a mathematical distribution that fits our data.

- **Explicit Density Estimation**
 - The neural network explicitly defines and computes the probability density function $P_{\theta}(X)$.

- **Implicit Density Estimation**
 - The neural network does not define an exact mathematical formula for $P_{\theta}(X)$.

2. Explicit Density Estimation



What we
will study

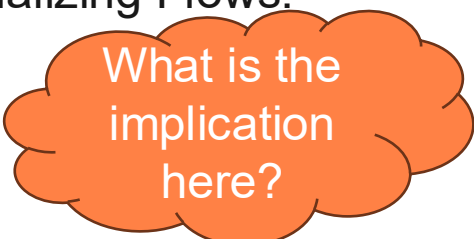
Explicit Density Estimation

- Formulate a model where $P_{\theta}(X)$ can be directly computed, allowing us to train the network by maximizing the log-likelihood of the training data.

- Tractable:

- The likelihood function can be computed and optimized exactly.
- *Examples:* Autoregressive models (e.g., PixelRNN, PixelCNN) and Normalizing Flows.

$$P(x_1, x_2, \dots, x_n) = \prod_{i=1}^n P(x_i | x_1, \dots, x_{i-1})$$



What is the implication here?

- Approximate:

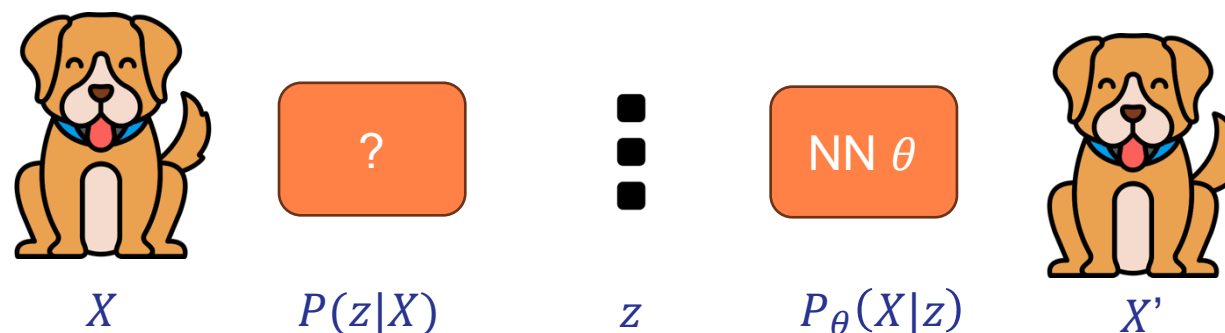
- The exact likelihood is computationally intractable, so we must optimize an approximation or a lower bound.
- *Examples:* Variational Autoencoders (VAEs).

Explicit Density Estimation - Approximate

- **The Generative Assumption:** Complex observed data X is generated from unobserved, continuous latent variables z . Our $P_{\theta}(X|z)$ model.
- **The Marginal Likelihood:** To evaluate the exact probability of our data $P_{\theta}(X)$, we must account for every possible configuration of z by integrating:

$$P_{\theta}(X) = \int P_{\theta}(X|z)P(z)dz$$

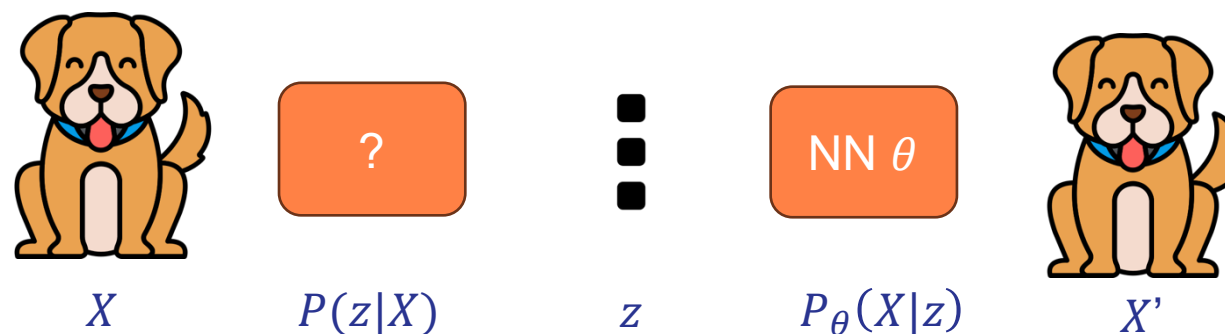
- **How can we train $P_{\theta}(X|z)$?**



Explicit Density Estimation - Approximate

- To train $P_{\theta}(X|z)$, we must be able to map our data backwards into the latent space. We need to compute the **true posterior**: $P(z|X)$

How can we do this?



Explicit Density Estimation - Approximate

- To train $P_\theta(X|z)$, we must be able to map our data backwards into the latent space. We need to compute the **true posterior**: $P(z|X)$

Bayes
again 😊

$$P(z|X) = \frac{P_\theta(X|z)P(z)}{P_\theta(X)} \Leftrightarrow \frac{P_\theta(X|z)P(z)}{\int P_\theta(X|z)P(z)dz}$$

Explicit Density Estimation - Approximate

- To train $P_\theta(X|z)$, we must be able to map our data backwards into the latent space. We need to compute the **true posterior**: $P(z|X)$

Bayes
again 😊

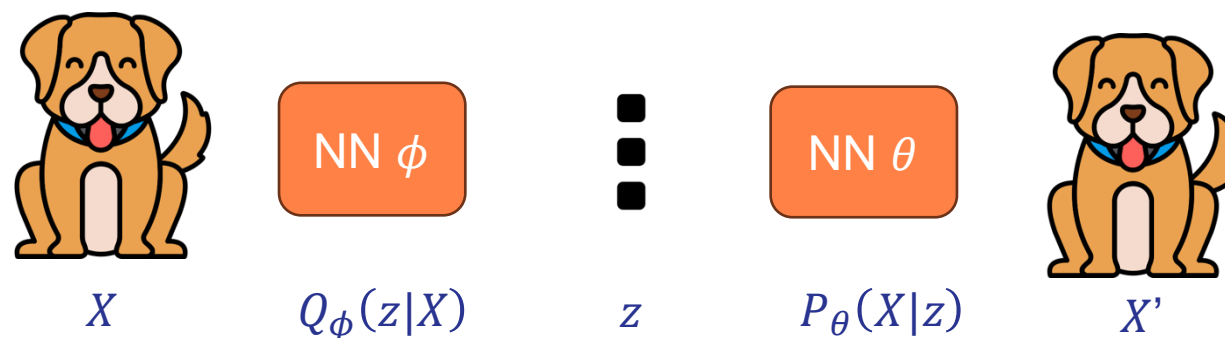
$$P(z|X) = \frac{P_\theta(X|z)P(z)}{P_\theta(X)} \Leftrightarrow \frac{P_\theta(X|z)P(z)}{\int P_\theta(X|z)P(z)dz}$$

- **Problem:** In deep learning, z is highly dimensional and $P_\theta(X|z)$ is a highly non-linear neural network.
 - The integral in the denominator has no analytical solution ☹.

Solution?

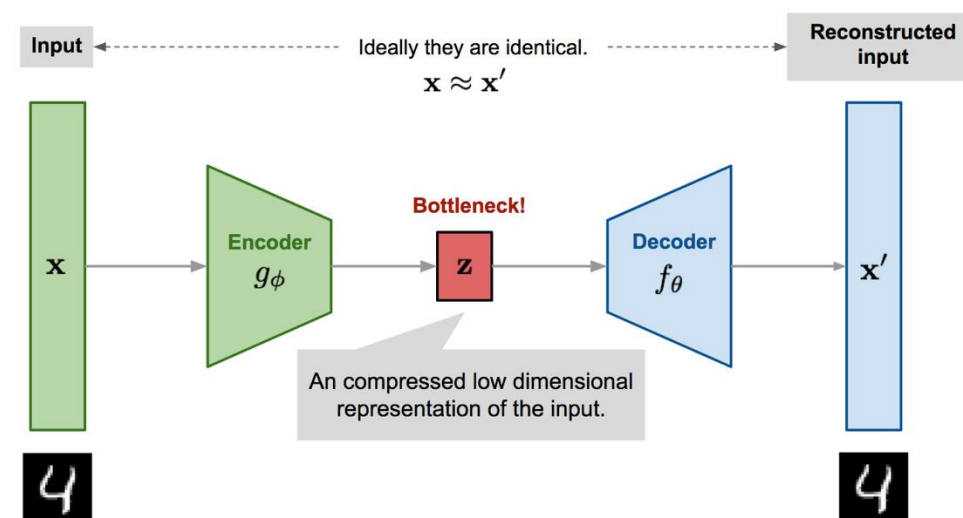
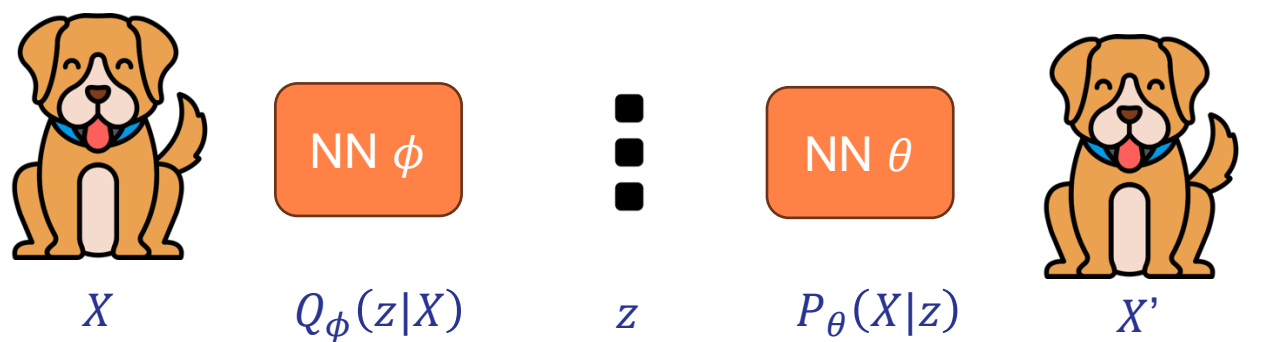
Explicit Density Estimation - Example

- We use another neural network to approximate $P(z|X)$.
- We introduce a **second** neural network to act as our approximation: $Q_\phi(z|X)$.
- The architecture:
 - Encoder ($Q_\phi(z|X)$): Looks at an image X and guesses the latent variables z .
 - Decoder ($P_\theta(X|z)$): Takes those latent variables z and reconstructs the image X



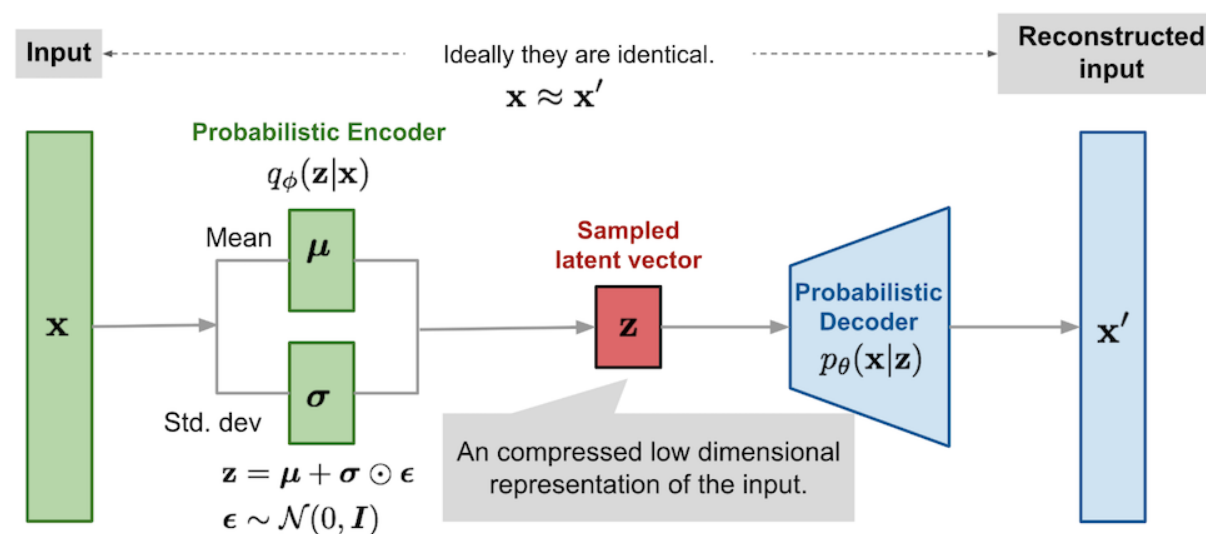
Autoencoder vs Variational Autoencoder

- By approximating our intractable math problem, we accidentally built an **Autoencoder** architecture! But there is a crucial difference:
- Autoencoder: Data compression and reconstruction
 - The latent space is discontinuous, which is no good for generation.
- Variation Autoencoder: Density estimation and generation
 - Latent space represents is continuous an represents a smooth distribution.



Variational Autoencoder

- Instead of outputting a single vector, the encoder neural network outputs **two vectors** of the same size to define a Gaussian distribution:

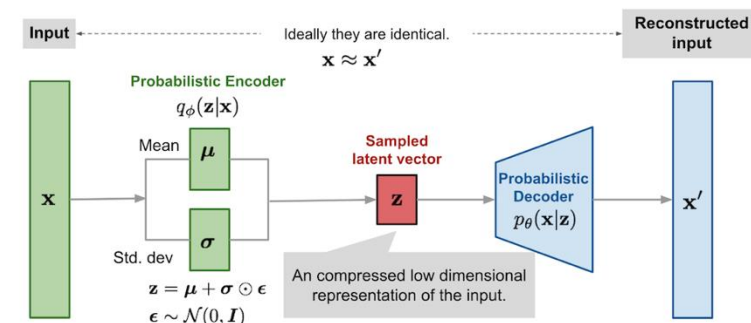


VAE Train

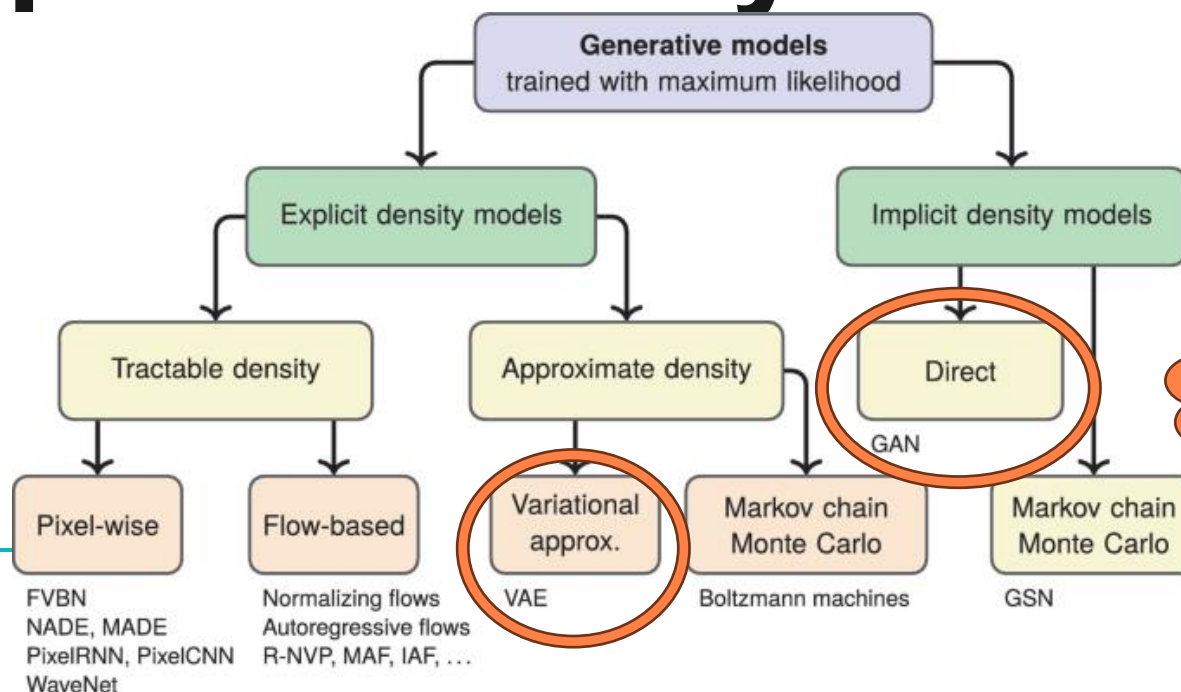
- Step 1: Pass the real input image X through the Probabilistic Encoder to compute the distribution parameters: mean μ and standard deviation σ .
- Step 2: Sample random noise $\epsilon \sim \mathcal{N}(0, I)$, and calculate the latent vector z deterministically: $z = \mu + \sigma \odot \epsilon$
- Step 3: Pass the sampled latent vector z through the Probabilistic Decoder to generate the reconstructed image X' .
- Step 4: Compare if X and X' are similar and if μ σ are close to a normal distribution $\mathcal{N}(0, I)$.

WHYnot ?
 $z \sim \mathcal{N}(\mu, \sigma)$

$$Loss = \frac{1}{N} \sum_i \underbrace{-\mathbb{E}_{z \sim q(z|x_i)} [\log_e p(x_i|z)]}_{\mathcal{L}_1} + \underbrace{KL(q(z|x_i) || p(z))}_{\mathcal{L}_2}$$



3. Implicit Density Estimation



What we
will study

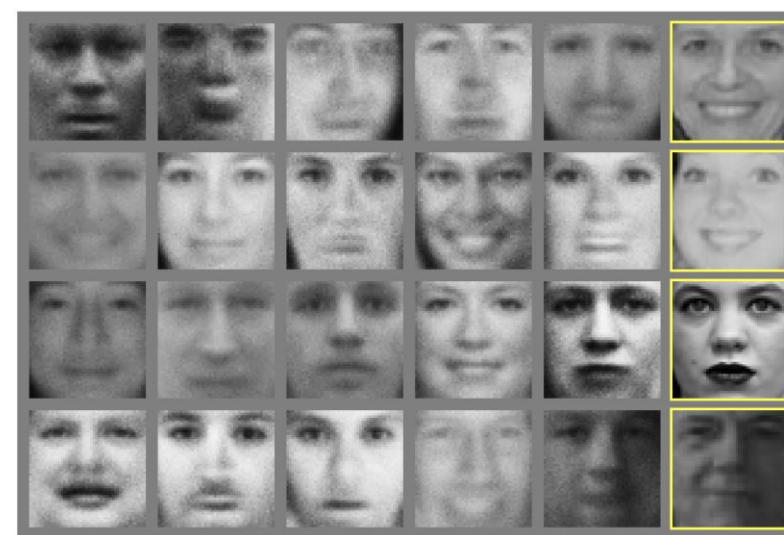
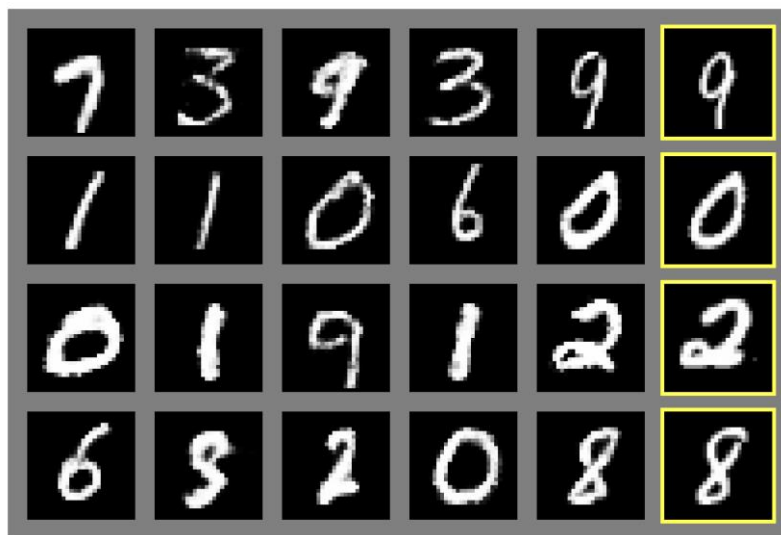
Implicit Density Estimation

- Learn a model that can sample from the distribution without ever defining $P(X)$ explicitly.
- **The goal:** we just want a "black box" where we put in noise and get out a realistic image.
- **Example:** Generative Adversarial Networks



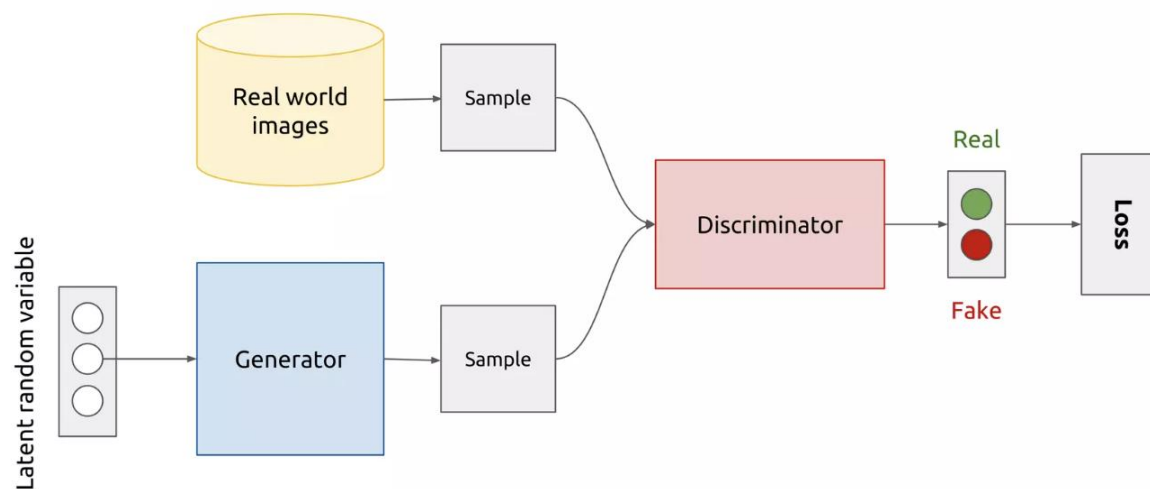
Implicit Density Estimation - GAN

- It were proposed by Ian Goodfellow in 2014 in a bar in Montreal with colleagues.
- His colleagues were struggling with complex mathematical models called Boltzmann Machines to generate images, but the math was incredibly heavy and the results were blurry.
- Ian proposed a radical shift in thinking: instead of trying to define the math of the distribution, why not let two neural networks teach each other through competition?



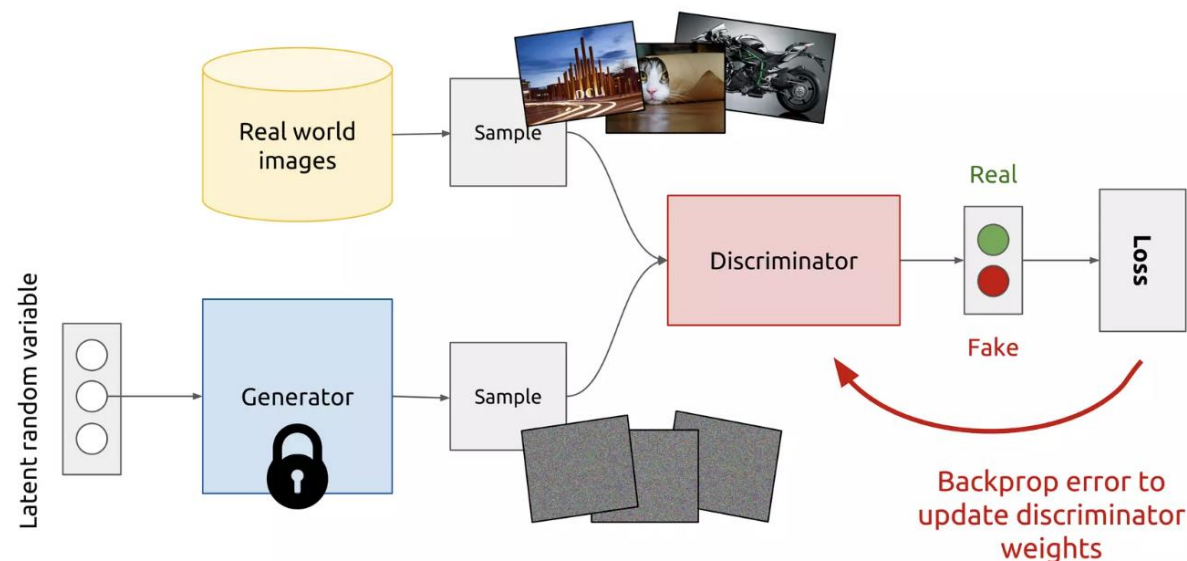
Understanding GANs

- Framed as a min-max game between two networks the Discriminator and the Generator.
 - Goal of the generator is to fool the discriminator
 - Goal of the discriminator is to catch the fake samples from the generator



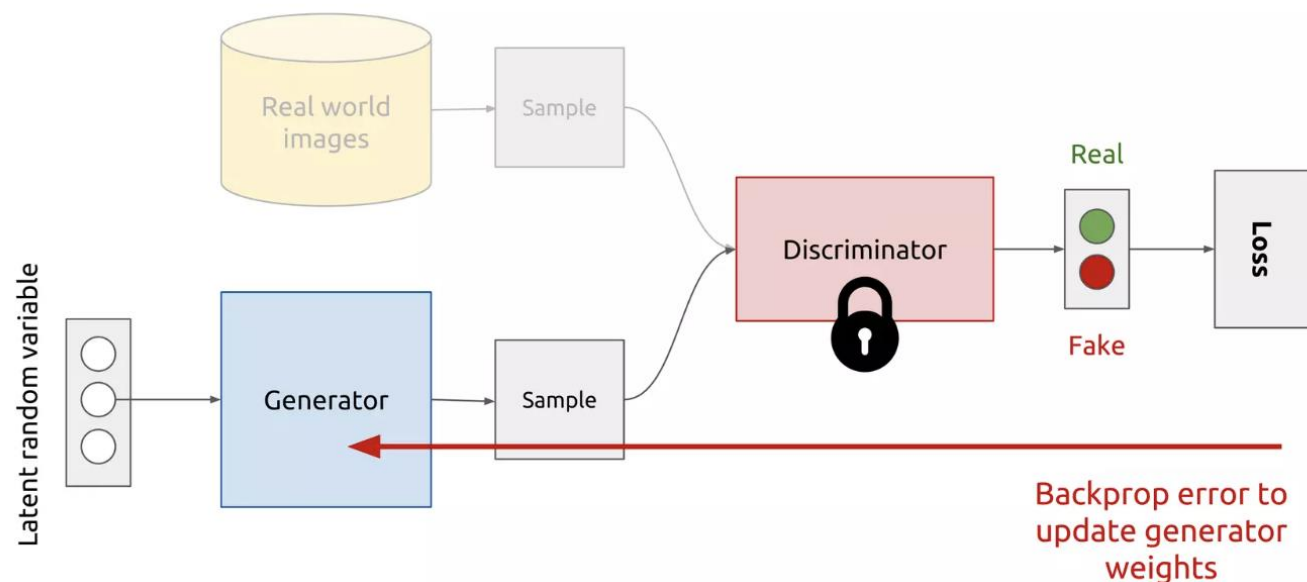
Understanding GANs

- Unlike the VAE, which uses a mathematical lower bound (ELBO), a GAN is trained through a min-max objective function.
- The Discriminator tries to maximize its accuracy, wanting to output a "1" for real images and a "0" for fakes.



Understanding GANs

- At the same time, the Generator is trying to minimize the Discriminator's accuracy by forcing it to output a "1" for the fakes it creates.



GAN – Loss

$$\min_G \max_D \mathbb{E}_{x \sim P_{real}(x)} [\log D(x)] + \mathbb{E}_{z \sim P_Z(z)} [\log(1 - D(G(z)))]$$

- To train discriminator:

$$\max_D \mathbb{E}_{x \sim P_{real}(x)} [\log D(x)] + \mathbb{E}_{z \sim P_Z(z)} [\log(1 - D(G(z)))]$$

- To train generator

$$\min_G \mathbb{E}_{z \sim P_Z(z)} [\log(1 - D(G(z)))]$$



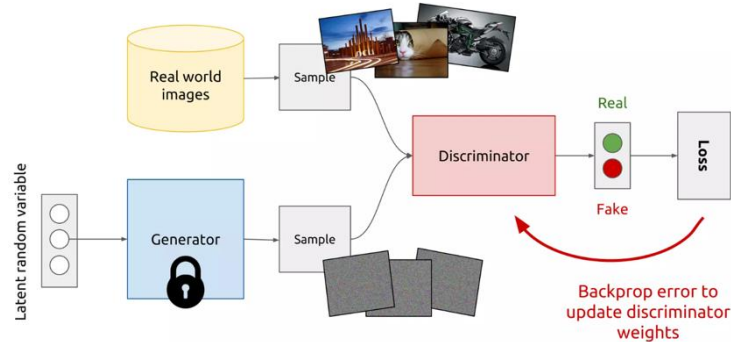
Trick, why?

$$\max_G \mathbb{E}_{z \sim P_Z(z)} [\log(D(G(z)))]$$

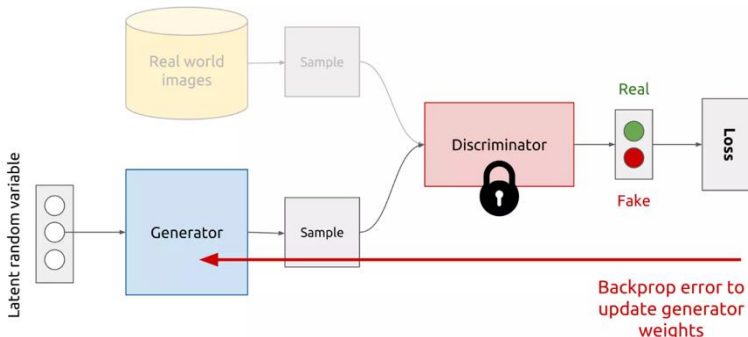
GAN - Hands on

➤ For loop

- Train **only** the discriminator with real and fake images.



- Train **only** the generator with the feedback from the discriminator.



```
# 1. Setup
D = Discriminator()
G = Generator()
criterion = nn.BCELoss() # Binary Cross Entropy
opt_D = torch.optim.Adam(D.parameters(), lr=0.0002)
opt_G = torch.optim.Adam(G.parameters(), lr=0.0002)

for epoch in range(epochs):
    for real_images in dataloader:
        batch_size = real_images.size(0)

        # Labels
        real_label = torch.ones(batch_size, 1) # Label = 1
        fake_label = torch.zeros(batch_size, 1) # Label = 0

        # -----
        # 2. Train Discriminator: max log(D(x)) + log(1 - D(G(z)))
        # -----
        D.zero_grad()

        # Part A: Real images
        output_real = D(real_images)
        loss_D_real = criterion(output_real, real_label) # log(D(x))

        # Part B: Fake images
        z = torch.randn(batch_size, latent_dim)
        fake_images = G(z)
        output_fake = D(fake_images.detach()) # .detach() so G isn't updated here
        loss_D_fake = criterion(output_fake, fake_label) # log(1 - D(G(z)))

        loss_D = loss_D_real + loss_D_fake
        loss_D.backward()
        opt_D.step()

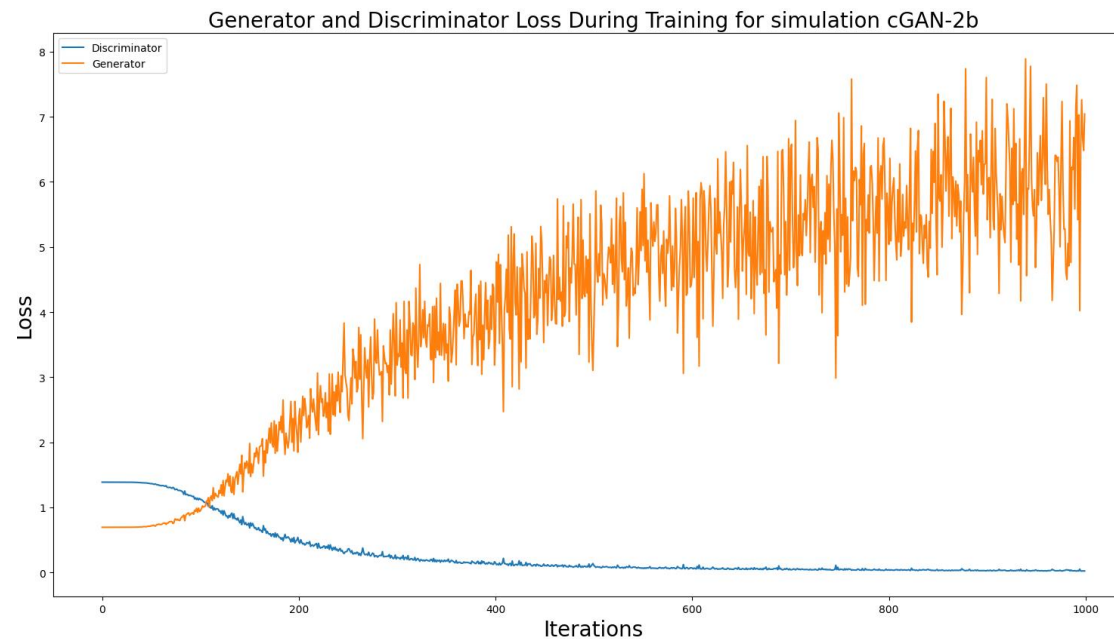
        # -----
        # 3. Train Generator: max log(D(G(z))) <-- THE TRICK
        # -----
        G.zero_grad()

        # We want the Discriminator to think these are REAL (label=1)
        output_fake_for_G = D(fake_images)
        loss_G = criterion(output_fake_for_G, real_label) # This is log(D(G(z)))

        loss_G.backward()
        opt_G.step()
```

GANs - Problems

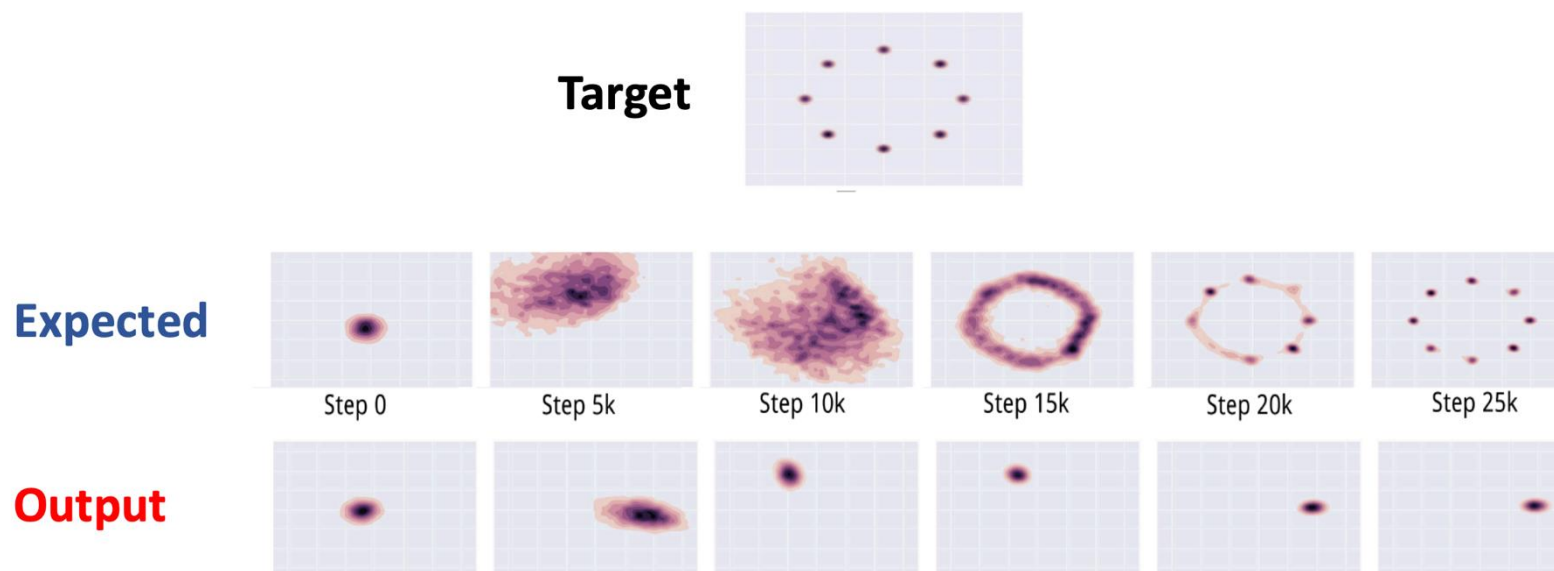
- Training is hard, non-convergence.
 - Our aim is to reach Nash equilibrium but there is no guarantee we will arrive there.



GANs - Problems

➤ Mode-collapse.

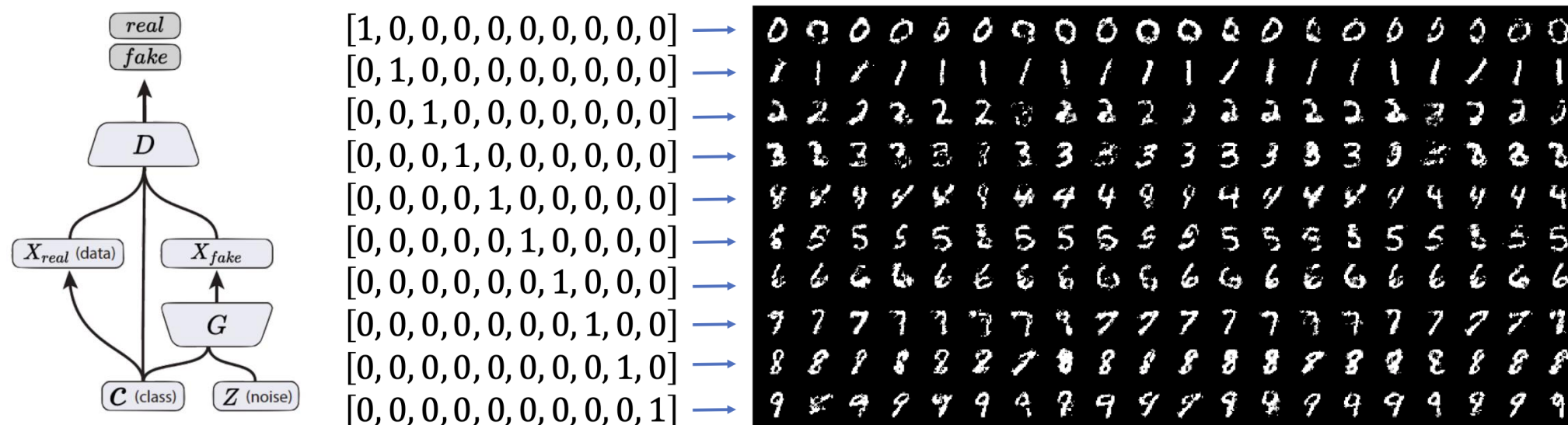
- The Generator finds one "safe" image that always fools the Discriminator and stops trying to learn anything else.



GANs - Evolutions

➤ Conditional GAN (cGAN)

- Instead of just generating a random image, cGANs allow you to provide a **label** (like "cat" or "dog") as an input to both the Generator and Discriminator.
- Shifted GANs from random generation to **controllable generation**.



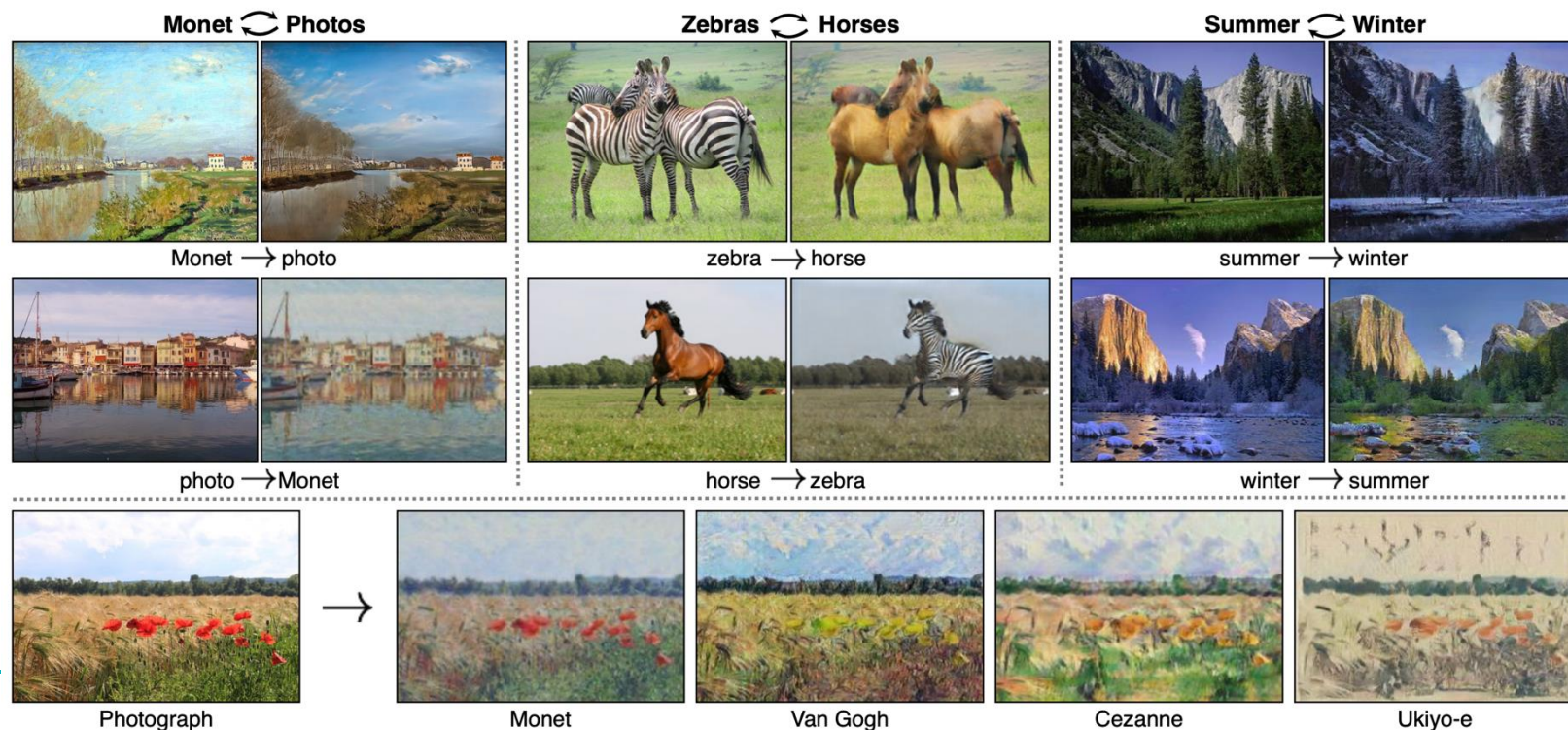
GANs - Evolutions

- 2017 – Wasserstein GAN
 - Addresses the mode collapse problem by changing the math behind the loss function to use Earth Mover's Distance.

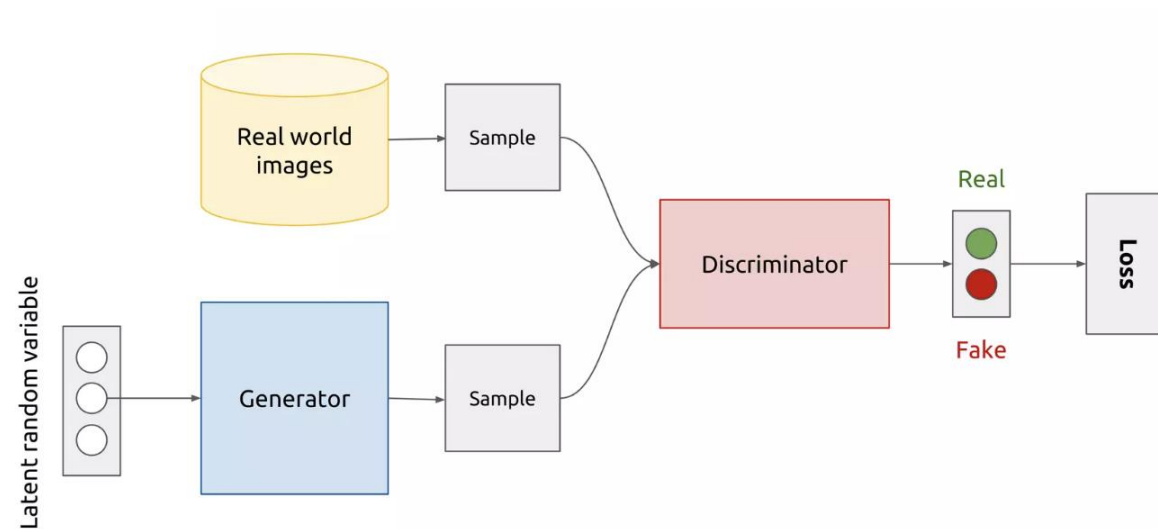
GANs - Evolutions

➤ 2017 – CycleGAN

- CycleGAN allowed for "Image-to-Image translation" without needing paired examples. For example, it can learn to turn a photo of a horse into a zebra without ever seeing a "horse-zebra" pair.



What about text?





15 minutes

Hands-On Lab

Thank You!

See you next week!

Lab submission due in 1 week
